IoT BASED REAL-TIME POLLUTION MONITORING SYSTEM

PROJECT PHASE I REPORT

Submitted to

The APJ Abdul Kalam Technological University

in partial fulfilment of the requirements for the award of the Degree of

Bachelor of Technology

in

Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

Submitted by

AKASH KRISHNA	**SBC22AM011**
AMISHA. P	**SBC22AM014**
G. SREEDEV	**SBC22AM027**
RITHUNANDHA K P	**SBC22AM048**

Under the Guidance of

Dr. DIVYA MOHAN

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)
SREE BUDDHA COLLEGE OF ENGINEERING, PATTOOR
(AUTONOMOUS)

OCTOBER, 2025

DECLARATION

We undersigned hereby declare that the project report **IoT BASED REAL-TIME POLLUTION MONITORING SYSTEM** submitted for partial fulfilment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under the supervision of Dr. Divya Mohan (Associate Professors, Department of Computer Science and Engineering). This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place : Pattoor

Date : 11-11-2025

AKASH KRISHNA

AMISHA P

G. SREEDEV

RITHUNANDHA K P

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

SREE BUDDHA COLLEGE OF ENGINEERING, PATTOOR P.O (AUTONOMOUS)
ALAPPUZHA

CERTIFICATE

This is to certify that this project phase one report entitled **"IoT BASED REAL-TIME POLLUTION MONITORING SYSTEM "** is a Bonafide record of the Project done by **AKASH KRISHNA (SBC22AM027), AMISHA. P (SBC22AM014), G. SREEDEV (SBC22AM027), RITHUNANDHA K P (SBC22AM048)** under our guidance and supervision towards partial fulfilment of the requirements for the award of Bachelor of Technology Degree in Computer Science and Engineering (Artificial Intelligence and Machine Learning) of the APJ Abdul Kalam Technological University during the year 2025.

Ms. LEKSHMI V.S
&
Ms. DIVYA. MK
Project Coordinator
ASSISTANT PROFESSORS
Dept. of CSE (AI & ML)

Dr. SUNIL S.S
Head of the Department
ASSOCIATE PROFESSOR
Dept. of CSE (AI&ML)

Dr. Divya Mohan
Project Supervisor
ASSOSIATE PROFESSOR
Dept. of CSE

ACKNOWLEDGMENT

We express my sincere gratitude to our respected principal **Dr. K KRISHNAKUMAR**, for his valuable support and advice.

We express my sincere thanks to **Dr. SUNIL S.S**, Head of the Department, Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Sree Buddha College of Engineering, Pattoor, for his valuable guidance and suggestions throughout the entire work.

We would like to thank my project coordinators, **Ms. LEKSHMI V.S & Ms. DIVYA M.K**, Assistant Professors, Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Sree Buddha College of Engineering, Pattoor, for the guidance and advice during the work.

We would like to thank with deep sense of gratitude and obligation to our project supervisor, **Dr. DIVYA MOHAN**, Associate Professors, Department of Computer Science and Engineering, Sree Buddha College of Engineering, Pattoor, for his valuable guidance and suggestions throughout the entire project work.

We express my sincere gratitude to all the members of the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Sree Buddha College of Engineering, Pattoor, for their encouragement and valuable assistance.

In particular, We are thankful to all those who have helped us directly or indirectly in completing this work. Above all I thank Almighty for giving me the health and strength to complete the work on time.

AKASH KRISHNA

AMISHA. P

G. SREEDEV

RITHUNANDHA K P

ABSTRACT

Air pollution continues to pose a serious environmental and health challenge, particularly in urban and semi-urban regions. Traditional monitoring systems often rely on fixed stations and fail to provide real-time, location-based insights. To address this gap, this project proposes a mobileintegrated IoT air quality monitoring system that collects, processes, analyzes, and visualizes environmental data to promote public awareness and informed decision-making. The system integrates a range of sensors — MQ135 for CO_2 and VOCs, MQ7 for carbon monoxide, PMS5003 for particulate matter (PM2.5 and PM10), and DHT22 for temperature and humidity — all connected to an Arduino microcontroller. A data cable enables data transmission to a connected laptop, which acts as the local server for real-time computation, storage, and analytical processing. The system determines geographic location through the connected device, integrating sensor data structured in JSON format for further analysis. Machine learning algorithms such as K-Means clustering and Random Forest can be implemented on the laptop to identify pollution hotspots and predict future air quality trends

A cross-platform mobile application developed using Flutter provides users with several key features, including:

1. Personalized user profiles with health condition tracking,
2. Real-time air quality monitoring of frequently visited areas.

Keywords: Air Pollution Monitoring, Internet of Things (IoT), Arduino Microcontroller, Air Quality Index (AQI), Wi-Fi Module, Sensor Integration, Machine Learning, K-Means Clustering, Random Forest, Flutter Application, Real-time Data Visualization, Pollution Hotspots, Community Reporting.

TABLE OF CONTENTS

ACKNOWLEDGEMENT I

ABSTRACT II

LIST OF TABLES VI

LIST OF FIGURES VI

LIST OF ABBREVIATIONS VI

CHAPTER 1 INTRODUCTION 1

1.1 Overview 1

1.2 Objective 2

1.3 Scheme of Report 2

CHAPTER 2 LITERATURE SURVEY 4

2.1 An IoT System for Air Pollution Monitoring with Safe Data Transmission 4

2.2 Deep-Learning Based Multi-Timestamp Multi-Location PM2.5 Prediction 5

2.3 Air Quality and Healthy Ageing: Predictive Modeling of Pollutants Using CNN Quantum-LSTM 7

2.4 Real-Time IoT Powered AI System for Monitoring and Forecasting of Air Pollution in Industrial Environment 8

2.5 A UAV-Based Real-Time Air Quality Monitoring and Prediction Using Machine Learning Approach 9

CHAPTER 3 DESIGN AND ARCHITECTURE 11

3.1 System Architecture Diagram 11

3.2 Flow of System Operation 12

3.3 Microcontroller Layer (Arduino) 12

3.4 Backend Processing Layer 12

3.5 Machine Learning Module 13

3.6 Mobile Application Layer 13

3.7 User Interaction Layer 13

CHAPTER 4 METHODOLOGY 14

4.1 Environmental Data Acquisition 14

4.2 Sensor Calibration and Initialization 15

4.3 Data Acquisition Using Arduino 15

4.4 Data Transmission and Communication 16

4.5 Data Preprocessing and Cleaning 16

4.6 Machine Learning-Based Pollution Prediction 17

4.7 Backend Processing and API Development 17

4.8 Mobile Application Data Visualization 18

4.9 Real-Time Monitoring and Alerts 18

4.10 Overall Workflow of the Methodology 18

CHAPTER 5 IMPLEMENTATION 20

5.1 Hardware Integration 20

5.2 Arduino Programming and Sensor Data Collection 20

5.3 Backend Server Development 21

5.4 Machine Learning Model Integration 22

5.5 Mobile Application Implementation 22
5.6 System Integration 23

CHAPTER 6 SYSTEM WORKFLOW & USE CASES 24
6.1 System Workflow 24
6.2 Use Case Scenarios 27

CHAPTER 7 COMPARISON WITH EXISTING SYSTEMS 28
7.1 Comparative Analysis 29

CHAPTER 8 TESTING 31
8.1 Hardware Testing 31
8.2 Communication Testing 31
8.3 Backend System Testing 31
8.4 Machine Learning Model Testing 32
8.5 Mobile Application Testing 32
8.6 System Integration Testing 32
CHAPTER 9 RESULTS 34
9.1 Circuit Design and Hardware Setup 34
9.2 Sensor Setup and Data Acquisition 35
9.3 Mobile Application Interface 35
9.4 Real-Time Pollution Monitoring Dashboard 36
9.5 Machine Learning Model Performance 37
9.6 Overall System Performance 38

CHAPTER 10 CONCLUSION 39

CHAPTER 11 FUTURE SCOPE 40

REFERENCES 41

LIST OF TABLES

NO.	NAME	PAGE
7.1	Feature Comparison of Our System with Competitors	28

LIST OF FIGURES

NO.	NAME	PAGE
3.1	Proposed System Architecture	11
5.1	System Workflow	25

LIST OF ABBREVIATIONS

i	AQI	Air Quality Index
ii	ML	Machine Learning
iii	NLP	Natural Language Processing
iv	CNN	Convolutional Neural Network
v	RNN	Recurrent Neural Network
vi	GAN	Generative Adversarial Network
vii	JSON	JavaScript Object Notation
vii	MQ-135	MQ-135 gas sensor
ix	LSTM	Long Short-Term Memory
x	PM2.5	Particulate Matter with diameter ≤ 2.5 µm
xi	PM10	Particulate Matter with diameter ≤ 10 µm

CHAPTER 1
INTRODUCTION

1.1 Overview

Air pollution has emerged as one of the most pressing global challenges of the 21st century, significantly impacting public health, ecosystems, and overall quality of life. Rapid industrialization, urban growth, and increased vehicular emissions have led to the accumulation of harmful pollutants in outdoor environments, particularly in densely populated urban areas. According to the World Health Organization (WHO), air pollution is responsible for more than 7 million premature deaths annually worldwide, with indoor air pollution accounting for approximately 4.1% of global mortality rates.

Air pollution is characterized by elevated levels of particulate matter (PM2.5 and PM10), nitrogen oxides (NOx), carbon monoxide (CO), ozone (O_3), sulfur dioxide (SO_2), and volatile organic compounds (VOCs). These pollutants are strongly associated with respiratory and cardiovascular diseases, premature mortality, and reduced life expectancy. Traditional environmental monitoring methods largely depend on static, government-operated stations equipped with sophisticated but expensive instruments. While these stations offer high accuracy, their limited number and fixed locations often fail to capture the spatial and temporal variability of pollutants across a city or region.

The advent of Internet of Things (IoT) technology has revolutionized environmental monitoring by enabling the deployment of low-cost, portable sensor networks throughout urban environments. IoT refers to interconnected devices embedded with sensors, processors, and communication modules capable of autonomously collecting and transmitting data. By deploying networks of IoT sensors, it becomes feasible to monitor pollution levels in real-time with high spatial resolution, capturing localized pollution events that traditional monitoring stations may miss.

This project proposes a comprehensive IoT-based real-time pollution monitoring system that integrates multiple environmental sensors with cloud-based analytics and a user-friendly mobile application. The system aims to provide continuous, accurate, and actionable air quality data to individuals, policymakers, and environmental agencies, enabling proactive responses to pollution incidents and supporting informed decision-making for public health protection.

1.2 Objectives

The primary objective of this project is to develop an intelligent system capable of monitoring environmental pollution in real time using Internet of Things (IoT) technologies and machine learning techniques. The system aims to provide accessible and reliable air quality information to users through a mobile application interface. The specific objectives of the proposed system are as follows:

1. To design and implement an IoT-based air quality monitoring system capable of collecting environmental parameters such as temperature, humidity, and gas concentrations.
2. To integrate environmental sensors including **MQ135**, **MQ7**, and **DHT22** with an Arduino microcontroller for continuous monitoring of air quality conditions.
3. To develop a data communication mechanism that transfers sensor readings from the Arduino device to a backend server using serial communication.
4. To implement a backend processing system using **Python** and **Flask** for real-time data processing, storage, and analysis.
5. To apply machine learning algorithms such as **Random Forest** to analyze environmental data and predict pollution conditions.
6. To develop a cross-platform mobile application using Flutter that enables users to view real-time air quality data and pollution predictions.
7. To provide visualization and alerts for unsafe pollution levels, helping users make informed decisions about their environment.
8. To create a low-cost and scalable environmental monitoring solution that can be deployed in urban and semi-urban areas.

1.3 Scheme of Report

CHAPTER 1: Provides an introduction to air pollution challenges, the role of IoT in environmental monitoring, and outlines the objectives and scope of the proposed system.

CHAPTER 2: Presents a detailed literature survey examining existing air quality monitoring approaches, IoT-based systems, machine learning applications in pollution prediction, and secure data transmission protocols.

CHAPTER 3: Describes the system design and architecture, including hardware components, sensor specifications, communication protocols, and cloud infrastructure.

CHAPTER 4: Explains the methodology employed for data collection and preprocessing, sensor calibration, machine learning model training and validation, and system integration.

CHAPTER 5: Illustrates the system workflow through detailed use cases demonstrating real-world applications for urban residents, commuters, and health-conscious individuals.

CHAPTER 6: Provides a comprehensive comparison of the proposed system with existing commercial and research-based air quality monitoring solutions.

CHAPTER 7: Explores future enhancements including edge computing integration, expanded sensor networks, advanced predictive models, and integration with smart city infrastructure.

CHAPTER 8: Concludes the report by summarizing key contributions, highlighting the system's potential impact on public health and environmental management, and outlining recommendations for future research.

CHAPTER 2
LITERATURE SURVEY

2.1 An IoT System for Air Pollution Monitoring with Safe Data Transmission

Authors: Janusz Bobulski, Sabina Szymoniak, Kamila Pasternak (2024)

This seminal paper [1] presents an innovative IoT-based air quality monitoring system deployed on autonomous electric vehicles to collect spatially diverse pollution data across urban environments. The research addresses a critical gap in traditional monitoring approaches by combining mobile sensing platforms with secure data transmission protocols, offering both comprehensive coverage and data integrity.

The proposed system architecture consists of four distinct layers: (1) a data collection layer utilizing autonomous vehicles equipped with advanced multi-parameter sensors, (2) an end devices layer comprising smartphones and tablets for user access, (3) a central server layer for data aggregation and processing, and (4) a communication layer ensuring secure bidirectional data flow. The sensing equipment measures PM10, PM2.5, volatile organic compounds (VOCs), carbon monoxide (CO), nitrogen dioxide (NO_2), nitric oxide (NO), sulfur dioxide (SO_2), temperature, humidity, atmospheric pressure, and GPS coordinates at 1.5-meter accuracy.

A particularly innovative aspect of this work is the implementation of the Amelia protocol, modified specifically for securing communication between autonomous vehicles and central servers. The protocol employs a two-phase approach: registration phase for initial authentication and communication phase for ongoing secure data transmission. During registration, vehicles exchange encrypted identifiers and timestamps with the central server, which generates unique numeric identifiers and validates credentials. The communication phase uses asymmetric encryption for authentication and symmetric session keys for efficient data transfer, protecting against replay attacks, man-in-the-middle attacks, and impersonation attempts.

The communication infrastructure leverages the Message Queue Telemetry Transport (MQTT) protocol within a publish-subscribe architecture. Autonomous vehicles function simultaneously as Publishers (collecting and sharing data) and Semi-Brokers (distributing localized information to nearby users), while the central server acts as the Main-Broker coordinating the entire network. This hierarchical approach enables efficient data distribution without overwhelming the central infrastructure.

Field deployment using a Nissan Leaf electric vehicle—selected specifically to avoid introducing combustion-based pollutants during measurement—demonstrated the system's effectiveness across diverse urban zones in Poland. Results revealed significant correlations between atmospheric pressure and particulate matter concentrations, with thermal inversions under high pressure conditions trapping pollutants in lower atmospheric layers. The system successfully identified pollution hotspots and temporal patterns, presenting data through interactive heat maps and numerical displays accessible via mobile applications.

Pros

1. **Mobile Sensing Coverage:** Autonomous vehicle deployment enables dynamic, wide-area monitoring capturing spatial pollution variations impossible with fixed stations.
2. **Robust Security:** The adapted Amelia protocol provides end-to-end encryption, mutual authentication, and resistance to common attack vectors, ensuring data integrity
3. **Real-time Accessibility:** MQTT-based architecture delivers immediate data updates to subscribed users, facilitating timely health-protective actions.
4. **Comprehensive Multi-parameter Monitoring:** Integration of gas sensors, particulate detectors, and meteorological instruments provides holistic air quality assessment.

Cons

1. **Infrastructure Dependency:** Full automation relies on mature autonomous vehicle technology, which remains under development in most regions.
2. **Cost and Complexity:** Equipping vehicles with specialized sensing arrays and maintaining mobile sensor networks involves substantial initial investment and ongoing maintenance.
3. **Coverage Limitations:** Mobile monitoring provides excellent spatial diversity but may lack continuous temporal coverage at specific locations compared to fixed stations.
4. **Data Processing Burden:** High-frequency mobile sensing generates massive data volumes requiring significant cloud infrastructure and bandwidth.

2.2 Deep-Learning-Based Multi-Timestamp Multi-Location PM2.5 Prediction

Authors: Yu-Lun Chiang, Jen-Cheng Wang, Mu-Hwa Lee, An-Chi Liu, and Joe-Air Jiang (2024)

This study, published in the *IEEE Internet of Things Journal*, presents an advanced IoT-based deep learning framework for real-time PM2.5 prediction in metropolitan environments. The

research addresses the limitations of traditional hourly air quality prediction methods by introducing a **multi-timestamp, multi-location prediction model** capable of forecasting PM2.5 fluctuations within 6–30 minutes [2]. The primary motivation is to improve short-term air quality forecasting accuracy for urban residents, where quick alerts are crucial for health-sensitive groups such as asthmatics and the elderly.

The system integrates *Airbox* IoT stations distributed across Taipei to collect high-frequency particulate matter data. This information trains two recurrent neural network architectures -**Long Short-Term Memory (LSTM)** and **Gated Recurrent Unit (GRU)** -designed to capture both temporal dependencies and spatial variations in pollution concentration. The models were validated using data collected via a mobile IoT-enabled vehicle monitoring system, ensuring dynamic data acquisition across diverse city zones. Experimental results indicate an exceptional prediction accuracy, achieving **0.922 μg/m³ RMSE with 100% precision** for the LSTM model and **0.940 μg/m³ RMSE with 95.7% accuracy** for the GRU model.

This innovative implementation demonstrates the potential of AI-driven IoT frameworks in delivering hyper-local, short-range air quality forecasts. The study highlights the value of *real-time multivariate data streams* and shows how recurrent deep learning models outperform conventional deterministic simulation methods.

Pros:

1. Extremely high precision and low RMSE through optimized LSTM/GRU structures.
2. Integration of stationary and mobile IoT sensing for improved spatial coverage.
3. Application of time-series multi-timestep analysis (6–30 min), allowing predictive alerts.
4. Robust verification using field-gathered IoT vehicle monitoring data.

Cons:

1. Dependence on dense sensor distribution increases operational costs.
2. Limited scalability to rural or resource-scarce areas with low IoT penetration.
3. Purely data-driven approach ignores meteorological feedback coupling.
4. Short prediction intervals restrict use for medium- and long-range forecasts.

2.3 Air Quality and Healthy Ageing: Predictive Modeling of Pollutants Using CNN Quantum-LSTM

Authors: Fareena Naz, Muhammad Fahim, Adnan Ahmad Cheema, Bradley D. E. McNiven, Tuan-Vu Cao, Ruth Hunter, and Trung Q. Duong (2025)

This recent publication in *IEEE Access* introduces an innovative hybrid architecture, **AirVCQnet** [3], for pollutant forecasting that combines *Variational Mode Decomposition (VMD)*, *Convolutional Neural Networks (CNN)*, and *Quantum Long Short-Term Memory (QLSTM)* models. The study aligns environmental analytics with the global health agenda on **"Healthy Ageing"**, recognizing clean air as a prerequisite for longevity and wellbeing. Using multivariate atmospheric data from Northern Ireland, the model predicts five critical pollutants -**PM2.5, PM10, NO_2, SO_2, and O_3** -under the UK's Clean Air Strategy framework.

AirVCQnet leverages **quantum computation advantages** to enhance signal decomposition and manage high-dimensional temporal sequences. The framework uses VMD to extract pollutant-related frequency features, CNNs to perform local spatial feature extraction, and QLSTM units to capture nonlinear temporal dependencies. By integrating quantum encoding, the system achieves up to a **14% performance gain** compared to classical VMD-CNN-LSTM counterparts, validated through RMSE, MAE, and R^2 metrics. This study's holistic approach connects environmental intelligence with public health imperatives, providing insight into the relationship between pollution dynamics and ageing populations. It underscores quantum machine learning's capacity to optimize pollutant forecasting for sustainable smart cities while aligning with UN Sustainable Development Goals (SDGs) 3, 7, and 11.

Pros:

1. First application of **quantum LSTM** for pollutant time-series prediction.
2. Integrates environmental forecasting with *healthy ageing* objectives.
3. Demonstrates substantial accuracy improvement (up to 14%) over classical models.
4. Employs robust decomposition and hybrid deep learning to enhance signal fidelity.

Cons:

1. High computational demand and quantum simulation resource dependency.
2. Experimental data limited to Northern Ireland urban conditions.
3. Model interpretability reduced due to hybrid quantum architecture complexity.
4. Quantum component currently limited to emulator rather than true quantum hardware.

2.4 Real-Time IoT-Powered AI System for Monitoring and Forecasting of Air Pollution in Industrial Environment

Authors: Montaser N.A. Ramadan, Mohammed A.H. Ali, Shin Yee Khoo, Mohammad Alkhedher, and Mohammad Alherbawi (2024)

This *Elsevier Ecotoxicology & Environmental Safety* study presents a comprehensive **IoT–AI integrated system** for real-time air pollution monitoring in the **chrome plating industry** [4]. The research focuses on addressing occupational hazards from exposure to toxic pollutants such as hexavalent chromium, volatile organic compounds (VOCs), sulphur oxides, and particulate matter (PM). It introduces an industrial-grade monitoring architecture incorporating **Internet of Things sensors** and **artificial intelligence forecasting models** to predict and mitigate pollution episodes. The prototype deploys a network of low-cost gas and particulate sensors within the chrome plating facility, measuring CO_2, NH_3, SO_2, NO_2, CO, O_3, and particulate matter. The system transmits data via a **Narrowband IoT-GSM module** to cloud servers, where machine learning algorithms - specifically **Long Short-Term Memory (LSTM)**, **Random Forest**, and **Linear Regression** - forecast pollutant levels for the coming hours. Proactive actions such as exhaust fan activation are triggered autonomously when levels approach hazardous thresholds. The Random Forest model achieved $R^2 = 0.84$ for PM2.5 prediction accuracy.

This project advances industrial sustainability by establishing a closed-loop environmental monitoring control. It addresses the real-time data scarcity in prior industrial studies and validates sensor performance in an operational chrome plant in Istanbul, making it a pioneering implementation of predictive AI and IoT in resource-intensive manufacturing environments.

Pros:

1. Field-validated industrial deployment achieving high predictive accuracy ($R^2 = 0.84$).
2. Proactive mitigation through automated ventilation triggering mechanisms.
3. Integrates diverse sensors and AI models for multi-pollutant tracking.
4. Demonstrates scalable architecture suitable for multiple heavy industries.

Cons:

1. Sensor calibration drift may impact long-term reliability.
2. Limited test scope confined to chrome plating sector.
3. Requires high computational and communication resources for continuous prediction.
4. Potential latency due to GSM-based narrowband transmission in dense factory environments.

2.5 A UAV-Based Real-Time Air Quality Monitoring and Prediction Using Machine Learning Approach

Authors: Huan-Chia Hsu, Getaneh Berie Tarekegn, Mastewal Endeshaw Getnet, Kuo-Pin Yu, and Li-Chia Tai (2025)

This paper, published in the *IEEE Internet of Things Journal* (Vol. 12, No. 14, July 2025), introduces an autonomous Unmanned Aerial Vehicle (UAV)-based air quality monitoring and prediction (AQMP) system designed to overcome the limitations of traditional static and vehicle-mounted air monitoring infrastructures [5]. Fixed ground stations lack spatial granularity, and vehicle systems face mobility constraints in congested or inaccessible zones. By leveraging UAV mobility, IoT sensors, and deep learning algorithms, the study presents a flexible real-time framework to collect and forecast multi-pollutant air quality data at varying altitudes.

The UAV platform employs lightweight sensors to detect PM2.5, PM10, CO_2, CO, and O_3 concentrations, alongside environmental parameters such as temperature and humidity. These sensors, integrated via a Raspberry Pi 4B microcomputer, transmit data to a remote server using 5G MQTT wireless communication, ensuring low-latency, high-throughput IoT networking. The architecture follows a four-layered model:

1. Sensing Layer for real-time data acquisition through multi-gas sensors,
2. Network Connectivity Layer using MQTT over 5G for ultra-fast transmission,
3. Data Processing Layer for cloud storage and API-based management with PHP and MySQL, and
4. Application Layer providing graphical data visualization and live analysis via a web interface.

For predictive analysis, the model integrates a hybrid Attention-based CNN–GRU (AC-GRU) algorithm, which dynamically assigns adaptive weights to features, extracts spatial-temporal dependencies, and captures temporal patterns efficiently. Comparative tests against benchmark models such as LSTM, GRU, and Random Forest demonstrate significantly improved accuracy—achieving R^2 of 0.985 and RMSE of 0.135 in low-pollution areas and maintaining high reliability ($R^2 = 0.63$) in high-pollution environments. Real-world validation was conducted across Hsinchu and Yunlin Counties, Taiwan, confirming robust field applicability.

The UAV-based solution overcomes spatial and cost inefficiencies of static stations by enabling dynamic, high-resolution air quality mapping. Beyond monitoring, it provides near-instant forecasts (30-second horizon) that support rapid pollution mitigation decisions and health alerts.

Pros:

1. Fully autonomous UAV-IoT integration with high spatial and temporal coverage.
2. Advanced attention-driven CNN–GRU hybrid significantly enhances short-term pollutant prediction accuracy.
3. Real-time 5G-MQTT communication enables seamless, low-latency data transfer.
4. Web-based visualization ensures user accessibility and data transparency for environmental control agencies.

Cons:

1. Battery constraints (32-minute flight time) limit long continuous missions.
2. 5G infrastructure dependency may hinder deployment in rural zones.
3. Complex calibration procedures require co-location with certified AQM stations.
4. Predictive accuracy may degrade under extreme weather or turbulence interference.

CHAPTER 3
DESIGN AND ARCHITECTURE

The design and architecture of the proposed system focus on creating an efficient framework for collecting environmental data, processing it, and delivering meaningful insights to users in real time. The architecture integrates hardware sensors, a microcontroller, a backend processing server, and a mobile application interface. This layered structure ensures seamless communication between system components and enables real-time monitoring of environmental pollution.

The system is designed to continuously collect environmental parameters such as temperature, humidity, carbon monoxide concentration, and air quality levels using sensors connected to a microcontroller. These sensor readings are transmitted to a backend server where they are processed and analyzed using machine learning algorithms. The processed information is then displayed to users through a mobile application developed using Flutter.

According to the system design presented in the project documentation, the architecture follows a sequential data flow starting from the sensors and ending at the mobile application interface.

3.1 System Architecture Diagram

The overall architecture of the proposed IoT-based pollution monitoring system consists of multiple interconnected components that work together to collect, process, analyze, and visualize environmental data.

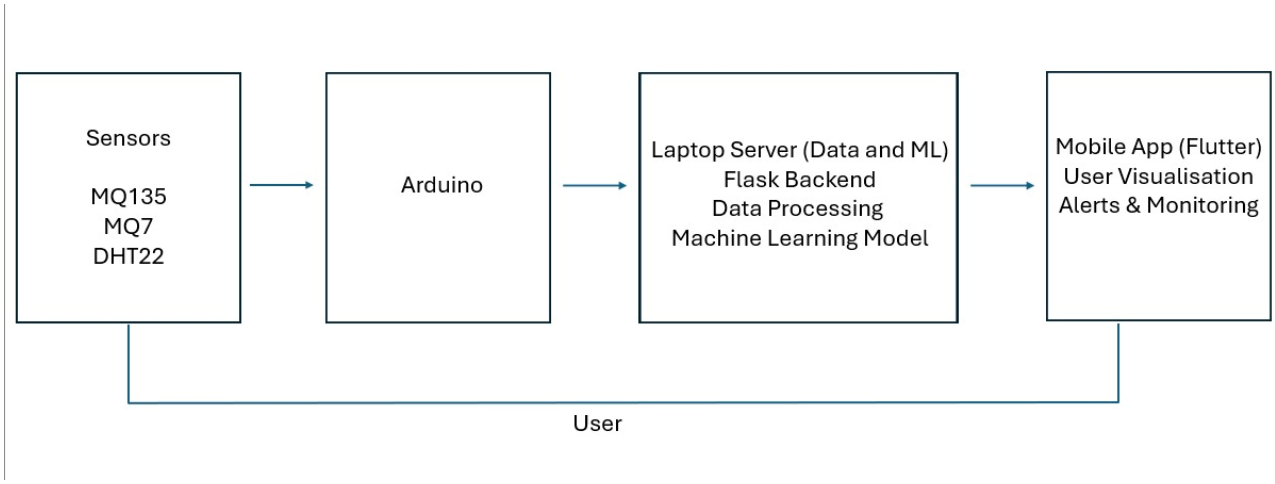

Fig 3.1 Proposed System Architecture

This architecture enables real-time monitoring and analysis of environmental pollution levels while providing user-friendly visualization through a mobile application.

3.2 Flow of System Operation

The workflow of the proposed system involves multiple sequential steps starting from environmental data collection and ending with visualization of pollution information on the mobile application.

1. Environmental sensors collect real-time environmental parameters such as temperature, humidity, carbon monoxide concentration, and air quality indicators.
2. The Arduino microcontroller reads the sensor values and converts them into structured data.
3. The collected sensor readings are transmitted to a laptop server through serial communication using a USB data cable.
4. The backend server processes the received sensor data and stores the most recent readings.
5. A machine learning model analyzes the sensor data and predicts the pollution condition or air quality status.
6. The processed data and prediction results are made available through a backend API.
7. The Flutter mobile application retrieves the latest data from the backend server and displays it in an interactive user interface.
8. Users can view real-time environmental information, pollution levels, and alerts regarding unsafe air quality conditions.

3.3 Microcontroller Layer (Arduino)

The Arduino microcontroller acts as the central data acquisition unit of the system. It is responsible for reading the sensor values and preparing the data for transmission. The Arduino program initializes the connected sensors and continuously collects readings at regular time intervals. Each sensor value is processed and formatted into a structured JSON object containing parameters such as temperature, humidity, carbon monoxide level, and air quality value. Once the data is structured, the Arduino sends the JSON-formatted sensor readings through serial communication to the connected laptop server. This process occurs continuously, allowing real-time monitoring of environmental conditions.

3.4 Backend Processing Layer

The backend processing layer is implemented using Python and the Flask web framework. This layer acts as the central processing unit of the system and performs multiple tasks including data acquisition, data processing, and prediction generation. The backend application reads incoming sensor data from the serial port connected to the Arduino device. A dedicated background thread continuously listens for new sensor readings to ensure uninterrupted data collection.

After receiving the data, the backend performs preprocessing operations such as filtering, adjusting sensor readings, and validating the received values. These steps help improve data reliability before

performing further analysis. The backend also integrates a machine learning model that predicts pollution conditions based on the sensor inputs. The model uses environmental parameters such as temperature, humidity, carbon monoxide levels, and air quality readings to determine whether the environment is safe or polluted. Finally, the processed data and prediction results are stored and made accessible through REST API endpoints.

3.5 Machine Learning Module

The machine learning module is responsible for analyzing environmental data and predicting pollution levels. The system utilizes a Random Forest classification model trained on historical environmental datasets.

The model takes sensor readings such as temperature, humidity, carbon monoxide concentration, and air quality values as input features. Based on these inputs, the model predicts pollution categories such as:

- Good
- Moderate
- Unhealthy

Random Forest was selected due to its robustness, ability to handle nonlinear relationships, and high classification accuracy. The trained model achieved strong predictive performance during evaluation and demonstrated reliable pollution classification capabilities.

3.6 Mobile Application Layer

The mobile application layer provides the user interface for interacting with the pollution monitoring system. The application is developed using the Flutter framework, which allows cross-platform deployment on Android devices. The mobile application communicates with the backend server through HTTP API requests to retrieve real-time environmental data. The retrieved information is displayed through a visually intuitive interface that includes environmental parameters such as temperature, humidity, carbon monoxide concentration, and air quality index. Additional features of the mobile application include user login functionality, location tracking, pollution analytics visualization, and real-time updates of air quality conditions.

The application also provides alerts to inform users when pollution levels exceed safe thresholds, enabling individuals to take precautionary actions.

3.7 User Interaction Layer

The final layer of the system is the user interaction layer. This layer represents the end users who interact with the mobile application to access environmental information.

Users can view pollution data for their current location, monitor real-time environmental conditions, and receive notifications regarding unsafe air quality levels. By providing easily accessible pollution information, the system helps users make informed decisions and reduce exposure to harmful

environmental conditions.

CHAPTER 4
METHODOLOGY

The methodology of the proposed IoT-based real-time pollution monitoring system describes the systematic process followed for designing, developing, and implementing the system. The methodology integrates Internet of Things (IoT) technologies, environmental sensing, data communication, backend processing, machine learning-based prediction, and mobile application visualization to provide a complete pollution monitoring solution. The overall process begins with environmental data collection through sensors connected to an Arduino microcontroller. The sensor readings are then transmitted to a backend server where the data is processed and analyzed. Machine learning techniques are applied to identify pollution conditions and predict air quality levels. The processed information is then delivered to a mobile application interface, allowing users to visualize environmental conditions and receive alerts when pollution levels exceed safe thresholds. The methodology ensures that environmental data is collected accurately, transmitted efficiently, analyzed intelligently, and presented to users in an accessible format.

4.1 Environmental Data Acquisition

The first stage of the methodology involves collecting environmental parameters from the surrounding environment using multiple sensors. Environmental monitoring requires capturing various atmospheric variables that influence air quality. For this purpose, the system uses a combination of gas sensors and environmental sensors to obtain relevant data.

The MQ135 gas sensor is used to detect harmful gases such as carbon dioxide, ammonia, benzene, and other volatile organic compounds that contribute to air pollution. This sensor provides a generalized measure of air quality by detecting changes in gas concentrations within the environment.

The MQ7 sensor is specifically designed to detect carbon monoxide (CO), a highly toxic gas commonly produced by vehicle exhaust emissions, industrial activities, and incomplete combustion processes. Monitoring carbon monoxide levels is crucial because exposure to high concentrations of CO can lead to severe health complications.

The DHT22 sensor is used to measure environmental temperature and humidity. Temperature and humidity significantly influence atmospheric conditions and may affect pollution dispersion and gas concentration levels. Including these parameters improves the accuracy of environmental analysis and predictive modeling.

Each sensor produces either analog or digital signals representing environmental conditions. These signals are captured by the Arduino microcontroller through its input pins. The sensors continuously collect environmental data at regular intervals, allowing the system to monitor pollution conditions in real time.

4.2 Sensor Calibration and Initialization

Before collecting environmental data, the sensors must be properly initialized and calibrated to ensure reliable measurements. Gas sensors such as MQ135 and MQ7 require an initial warm-up period before they can produce stable readings. This warm-up period allows the internal sensing elements of the sensors to stabilize and reach optimal operating conditions.

During the initialization stage, the Arduino program initializes the DHT22 temperature and humidity sensor using the appropriate sensor library. The microcontroller establishes communication with the sensor and prepares it to capture environmental readings.

Similarly, the MQ gas sensors are connected to the analog input pins of the Arduino board. The analog signals produced by these sensors correspond to variations in gas concentration levels in the surrounding environment. The Arduino reads these signals using its analog-to-digital converter (ADC), which converts analog voltage levels into digital values that can be processed by the system.

Once the sensors are initialized, the system begins continuous environmental monitoring by collecting readings at predefined time intervals.

4.3 Data Acquisition Using Arduino

The Arduino microcontroller acts as the central data acquisition unit in the system. Its primary responsibility is to read sensor values and convert them into structured data suitable for further processing.

The Arduino program periodically reads temperature and humidity values from the DHT22 sensor using the DHT library functions. These values represent environmental conditions that may influence air quality measurements.

At the same time, the Arduino reads analog values from the MQ135 and MQ7 sensors through its analog input pins. These readings correspond to gas concentration levels detected by the sensors.

After collecting sensor values, the Arduino program performs basic validation checks to ensure that the readings are not invalid or corrupted. For example, if the DHT22 sensor returns invalid readings such as "Not a Number" values, the program ignores these readings and waits for the next measurement cycle.

Once the sensor values are validated, the Arduino organizes them into a structured format. The readings are formatted as a JSON object containing the following parameters:

- Temperature
- Humidity
- Carbon monoxide level
- Air quality value

This structured data format allows easy transmission and processing of sensor readings in the backend system.

4.4 Data Transmission and Communication

After collecting and structuring the sensor data, the Arduino transmits the data to a connected laptop server using serial communication. Serial communication is a reliable and efficient method for transmitting data between microcontrollers and computers. The Arduino sends the JSON-formatted sensor readings through the USB serial interface at a predefined baud rate. Each transmission contains a complete set of environmental parameters, allowing the backend system to process the data in real time. On the server side, a Python-based backend application continuously listens for incoming serial data. The backend establishes a serial communication connection with the Arduino device using the appropriate communication port. A background thread is used to ensure that the backend can continuously read incoming sensor data without interrupting other server operations. This approach enables uninterrupted environmental monitoring and real-time data processing.
Once the backend receives a JSON string containing sensor readings, the system parses the data and converts it into a structured Python object for further analysis.

4.5 Data Preprocessing and Cleaning

Raw sensor readings often contain noise, fluctuations, or minor inconsistencies that must be addressed before performing analysis or prediction. Therefore, the methodology includes a preprocessing stage to improve data quality and reliability.

During preprocessing, the backend system validates the received sensor readings to ensure that all required parameters are present. If any sensor value is missing or invalid, the system discards the incomplete data and waits for the next reading cycle. The backend also performs minor adjustments to certain sensor values to remove potential measurement offsets. These adjustments help normalize the data and reduce sensor noise. Additionally, the system ensures that all sensor values remain within acceptable ranges. For example, negative gas concentration

values or unrealistic temperature readings are corrected or filtered out during preprocessing. The cleaned and validated data is then stored as the latest environmental reading and prepared for further analysis.

4.6 Machine Learning-Based Pollution Prediction

A key component of the proposed system is the integration of machine learning algorithms for pollution prediction. Machine learning techniques allow the system to analyze environmental data and identify pollution conditions based on patterns observed in historical datasets.

The system utilizes a Random Forest classification model trained on environmental data samples containing pollution-related parameters. Random Forest is an ensemble learning algorithm that constructs multiple decision trees and combines their outputs to produce a final prediction.

The model takes the following environmental parameters as input features:

- Temperature
- Humidity
- Carbon monoxide concentration
- Air quality sensor readings

Based on these inputs, the model predicts the pollution status of the environment. The output of the model may represent categories such as Good, Moderate, or Unhealthy air quality conditions.

Random Forest was selected for this system due to its advantages in handling complex datasets, reducing overfitting, and providing high predictive accuracy. The trained model demonstrated strong performance during evaluation and achieved a classification accuracy of over 90%.

4.7 Backend Processing and API Development

The backend server plays a central role in the methodology by coordinating data acquisition, processing, prediction, and communication with the mobile application. The backend application is implemented using Python and the Flask web framework. Flask provides a lightweight web server that enables the creation of RESTful API endpoints for communication with external applications. The backend continuously receives sensor data from the Arduino device, processes the data, and stores the most recent environmental readings. The machine learning model is applied to these readings to generate pollution predictions. The server exposes API endpoints that allow the mobile application to request the latest environmental data and prediction results. When the mobile application sends a request to the backend server, the API

returns a JSON response containing the most recent sensor readings and predicted pollution status. This API-based architecture ensures efficient communication between the backend server and the mobile application.

4.8 Mobile Application Data Visualization

The final stage of the methodology involves presenting the processed environmental data to users through a mobile application interface. The mobile application is developed using Flutter, a cross-platform framework that enables the creation of mobile applications for Android devices. The Flutter application communicates with the backend server using HTTP requests to retrieve real-time environmental data. The retrieved data is displayed through an interactive dashboard that presents various environmental parameters. The application interface allows users to view information such as temperature, humidity, carbon monoxide levels, and air quality index values. The application also displays the predicted pollution status based on machine learning analysis. Additionally, the mobile application integrates location-based features that allow users to monitor pollution conditions in their current location. This functionality enhances the usefulness of the system by providing personalized environmental information.

4.9 Real-Time Monitoring and Alerts

The system is designed to provide continuous real-time monitoring of environmental conditions. Sensor readings are collected at regular intervals and transmitted to the backend server for processing. Whenever pollution levels exceed predefined thresholds, the system generates alerts to notify users of potentially unsafe environmental conditions. These alerts help users take precautionary actions such as avoiding polluted areas or reducing outdoor exposure. Real-time monitoring ensures that environmental information remains up-to-date and allows users to respond quickly to changes in air quality conditions.

3.10 Overall Workflow of the Methodology

The complete workflow of the proposed pollution monitoring system can be summarized as follows:

1. Environmental sensors collect atmospheric data.
2. The Arduino microcontroller reads and processes sensor values.
3. Sensor data is structured into JSON format.
4. The data is transmitted to the backend server via serial communication.
5. The backend processes and cleans the sensor readings.

6. Machine learning algorithms analyze the data and predict pollution levels.
7. The processed data and predictions are stored on the server.
8. The mobile application retrieves the latest information through API requests.
9. Users view real-time pollution data and receive alerts when necessary.

This methodology ensures an efficient and reliable system capable of providing real-time environmental monitoring and predictive analysis of air quality conditions.

CHAPTER 5
IMPLEMENTATION

The implementation phase of the proposed IoT-based real-time pollution monitoring system involves the integration of hardware components, backend software modules, machine learning algorithms, and a mobile application interface. The goal of this phase is to transform the system design and methodology into a functional system capable of collecting environmental data, processing it in real time, and presenting meaningful insights to users.

The system implementation is divided into several stages including hardware integration, sensor programming using Arduino, backend server development, machine learning model integration, and the development of a mobile application interface using Flutter. Each component of the system was implemented in a modular manner to ensure flexibility, scalability, and ease of maintenance.

5.1 Hardware Integration

The hardware component of the system consists of environmental sensors connected to an Arduino microcontroller. The primary sensors used in the system include the MQ135 gas sensor, MQ7 carbon monoxide sensor, and DHT22 temperature and humidity sensor.

The MQ135 sensor is connected to one of the analog input pins of the Arduino board and is responsible for detecting harmful gases and general air quality conditions. Similarly, the MQ7 sensor is connected to another analog input pin to detect carbon monoxide concentrations in the surrounding environment. The DHT22 sensor, which measures temperature and humidity, is connected to a digital input pin of the Arduino microcontroller.

The Arduino board acts as the central interface between the sensors and the computing system. It collects electrical signals from the sensors and converts them into digital values using its built-in analog-to-digital converter. Proper wiring and power supply connections were established to ensure stable sensor operation and reliable data acquisition.

To ensure that the gas sensors provide accurate readings, the sensors were allowed to warm up for a short duration before starting the monitoring process. This warm-up period allows the sensor elements to stabilize and produce more reliable measurements.

5.2 Arduino Programming and Sensor Data Collection

The Arduino microcontroller was programmed using the Arduino IDE to control the sensors and collect environmental data. The program initializes the DHT22 sensor using the DHT sensor

library and configures the analog input pins for reading gas sensor values.

During operation, the Arduino continuously reads temperature and humidity values from the DHT22 sensor and obtains analog readings from the MQ135 and MQ7 sensors. These readings represent environmental parameters such as temperature, humidity, carbon monoxide concentration, and general air quality levels.

The Arduino program performs basic validation checks to ensure that the sensor readings are valid. If invalid values are detected, the system skips the current reading cycle and waits for the next measurement interval.

Once valid readings are obtained, the Arduino organizes the sensor values into a structured JSON format. This format contains key-value pairs representing each environmental parameter. Structuring the data in JSON format simplifies communication between the Arduino microcontroller and the backend processing system.

After formatting the data, the Arduino transmits the JSON object through the serial communication interface at a predefined baud rate. The transmission occurs at regular intervals to ensure continuous environmental monitoring.

5.3 Backend Server Development

The backend server plays a crucial role in receiving sensor data, processing it, and providing it to the mobile application. The backend of the system was developed using the Python programming language and the Flask web framework.

The backend server establishes serial communication with the Arduino device using the appropriate communication port. A dedicated background thread continuously listens for incoming data from the Arduino microcontroller. This threaded approach ensures that data acquisition runs independently of other backend operations.

When a JSON string containing sensor readings is received, the backend server parses the data and extracts environmental parameters such as temperature, humidity, carbon monoxide levels, and air quality indicators. These values are stored in a shared data structure that maintains the most recent environmental readings.

The backend server also performs preprocessing operations to clean the sensor data. These operations include removing noise, adjusting sensor offsets, and validating sensor readings to ensure they fall within acceptable ranges.

To enable communication with external applications, the Flask backend provides REST API

endpoints that allow clients to retrieve the latest environmental readings. These APIs return data in JSON format, making it easy for the mobile application to access and display the information.

5.4 Machine Learning Model Integration

A machine learning model was integrated into the backend system to analyze environmental data and predict pollution conditions. The system uses a trained Random Forest classification model to determine air quality levels based on sensor readings.

The trained machine learning model was stored as a serialized file and loaded into the backend server using Python's pickle library. When new sensor readings are received, the backend prepares the input features required by the model. These features include temperature, humidity, carbon monoxide concentration, and air quality sensor values.

The input data is passed to the machine learning model, which generates a prediction representing the pollution status of the environment. The predicted results are categorized into pollution levels such as Good, Moderate, or Unhealthy.

In situations where the trained model is not available, the system uses a fallback mechanism that generates approximate predictions based on weighted sensor values. This ensures that the system continues to function even if the machine learning model cannot be loaded.

The predicted pollution status is combined with the sensor readings and stored as part of the latest environmental data.

5.5 Mobile Application Implementation

The mobile application was developed using the Flutter framework, which enables the creation of cross-platform mobile applications using a single codebase. Flutter was selected due to its ability to produce high-performance applications with responsive user interfaces.

The mobile application communicates with the backend server using HTTP requests to retrieve environmental data from the REST API endpoints. The application periodically sends requests to the backend server and receives JSON responses containing sensor readings and pollution predictions.

The retrieved data is displayed through an interactive dashboard that presents environmental parameters in an intuitive and user-friendly format. The dashboard includes visual indicators, numerical displays, and status labels that help users quickly interpret air quality conditions.

The mobile application also includes features such as user authentication, location-based

monitoring, and alert notifications for unsafe pollution levels. These features enhance the usability of the system and provide personalized environmental information to users.

5.6 System Integration

The final stage of the implementation process involves integrating all system components into a complete working system. The hardware sensors, Arduino microcontroller, backend server, machine learning module, and mobile application were connected and tested to ensure seamless communication.

During system integration, the Arduino continuously transmitted sensor data to the backend server through serial communication. The backend processed the received data, applied machine learning predictions, and made the results available through API endpoints.

The Flutter mobile application successfully retrieved the environmental data from the backend server and displayed real-time pollution information to users. Continuous monitoring confirmed that the system was able to collect and display environmental data in real time.

Through the integration of hardware sensing devices, backend data processing modules, and mobile application technologies, the proposed system was successfully implemented as a complete IoT-based real-time pollution monitoring platform.

CHAPTER 6
SYSTEM WORKFLOW AND USE CASES

6.1 System Workflow

The following diagram illustrates the overall workflow of the proposed pollution monitoring system.

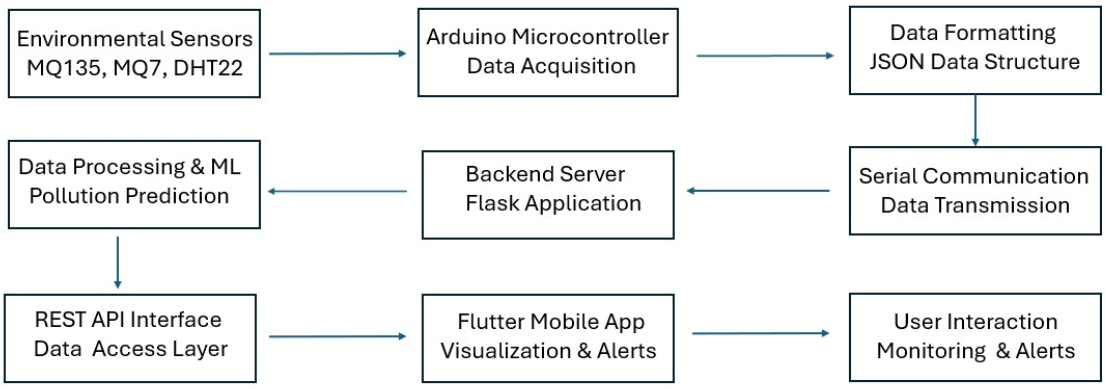

Fig 6.1 System Workflow

Step 1: Environmental Sensor Data Collection

The first stage in the system workflow involves collecting environmental data using sensors. Environmental sensors are responsible for detecting various atmospheric parameters that influence air quality.

The MQ135 gas sensor measures concentrations of gases such as carbon dioxide and volatile organic compounds. These gases are important indicators of environmental pollution and help determine the overall air quality.

The MQ7 sensor is specifically designed to detect carbon monoxide gas, which is one of the most dangerous pollutants produced by vehicle emissions and industrial activities.

The DHT22 sensor measures environmental temperature and humidity. These parameters are useful for analyzing atmospheric conditions and understanding pollution dispersion patterns.

The sensors continuously monitor environmental conditions and generate electrical signals corresponding to the measured parameters. These signals are transmitted to the Arduino microcontroller for further processing.

Step 2: Data Acquisition Using Arduino Microcontroller

The Arduino microcontroller serves as the central data acquisition unit in the system. It receives signals from the connected sensors and converts them into digital values that can be processed by software applications. The Arduino program periodically reads sensor values at predefined intervals. The temperature and humidity readings are obtained from the DHT22 sensor using dedicated library

functions. Similarly, analog readings from the MQ135 and MQ7 sensors are captured through the analog input pins of the microcontroller. After collecting the sensor readings, the Arduino validates the values to ensure that they are within acceptable ranges. Invalid readings are ignored to prevent inaccurate data from being transmitted to the backend server. Once the sensor values are verified, the Arduino prepares the data for transmission.

Step 3: Data Formatting and JSON Structure

Before transmitting sensor readings to the backend server, the Arduino organizes the collected data into a structured format. The system uses JSON (JavaScript Object Notation) to represent sensor readings in a structured and easily interpretable format. Each JSON object contains key-value pairs representing environmental parameters such as temperature, humidity, carbon monoxide concentration, and air quality values. Using JSON format offers several advantages, including simplicity, flexibility, and compatibility with various programming languages. The backend server can easily parse JSON objects and extract relevant sensor values for further analysis. The structured format ensures that data remains organized and reduces the complexity of communication between hardware and software components.

Step 4: Data Transmission through Serial Communication

Once the sensor data is formatted as JSON, it is transmitted to the backend server using serial communication. Serial communication provides a reliable mechanism for transferring data between the Arduino microcontroller and the laptop server. The Arduino sends sensor readings through a USB serial connection at a predefined communication speed known as the baud rate. Each data transmission contains a complete set of environmental parameters representing the current environmental conditions. On the receiving side, the backend server continuously listens for incoming serial data. A Python-based program establishes a serial connection with the Arduino device and reads incoming data streams. This continuous communication enables the system to receive real-time environmental data without interruption.

Step 5: Backend Server Data Processing

After receiving the sensor data, the backend server processes the information to prepare it for analysis and visualization. The backend server is implemented using Python and the Flask web framework. The server parses incoming JSON data and extracts environmental parameters such as temperature, humidity, carbon monoxide levels, and air quality indicators. These values are stored in a data structure that maintains the most recent environmental readings. The backend also performs preprocessing operations such as filtering noisy sensor readings and adjusting sensor

values when necessary. These preprocessing steps improve the reliability of the data before it is used for prediction or visualization. The processed data is then passed to the machine learning module for pollution analysis.

Step 6: Machine Learning-Based Pollution Prediction

The machine learning module analyzes environmental sensor data to determine pollution conditions. The system uses a trained Random Forest model to classify air quality conditions based on environmental parameters. The model takes temperature, humidity, carbon monoxide concentration, and air quality readings as input features. Using these inputs, the model predicts the pollution status of the environment. The predicted results are categorized into pollution levels such as Good, Moderate, or Unhealthy. These predictions help users understand whether the current environmental conditions are safe. The prediction results are stored along with sensor readings and are made available through the backend server.

Step 8: API-Based Data Access

To allow external applications to access environmental data, the backend server exposes REST API endpoints. These APIs enable the mobile application to retrieve real-time sensor readings and prediction results. When the mobile application sends a request to the server, the API responds with the latest environmental data in JSON format. This data includes sensor readings as well as the predicted pollution status. Using API-based communication allows the system to maintain a flexible architecture where multiple client applications can access the environmental data if required.

Step 9: Mobile Application Visualization

The Flutter mobile application acts as the user interface for the pollution monitoring system. It retrieves environmental data from the backend server through API requests and displays the information through an intuitive dashboard. The application presents environmental parameters such as temperature, humidity, carbon monoxide levels, and air quality values in a visually appealing format. Graphical indicators and numerical displays allow users to quickly understand the environmental conditions. The application also highlights pollution status using visual cues such as color-coded indicators or warning messages when pollution levels become unsafe. This real-time visualization helps users stay informed about environmental conditions in their surroundings.

Step 10: User Interaction

The final stage of the workflow involves user interaction with the mobile application. Users can access the application to monitor real-time environmental conditions and pollution levels. The application allows users to view air quality data for their current location and track pollution trends over time. In addition, users receive alerts or notifications when pollution levels exceed predefined thresholds. By providing accessible environmental information, the system empowers users to make informed decisions regarding outdoor activities and exposure to polluted environments.

6.2 Use Case Scenarios

Use Case 1: Real-Time Pollution Monitoring

Users can access the mobile application to view real-time environmental data collected from sensors. The application displays parameters such as temperature, humidity, carbon monoxide levels, and air quality indicators.

Use Case 2: Pollution Prediction

The machine learning module analyzes sensor data and predicts pollution levels. Users can view predicted air quality conditions through the mobile application dashboard.

Use Case 3: Location-Based Monitoring

The mobile application allows users to monitor pollution levels for their current location. The system integrates location information to provide relevant environmental data.

Use Case 4: Pollution Alerts

When pollution levels exceed safe limits, the system generates alerts to notify users about unsafe environmental conditions.

Use Case 5: Environmental Awareness

The system promotes awareness about environmental pollution by providing real-time air quality information to users. This enables individuals to make informed decisions regarding their health and safety.

CHAPTER 7
COMPARISON WITH EXISITING SYSTEM

The proposed IoT-based pollution monitoring system is evaluated against existing commercial and research platforms across multiple dimensions.

Feature	Proposed System	Government CPCB Stations	PurpleAir	Airveda	Google Air Quality
Real-time Monitoring	Yes (5-min intervals)	Yes (hourly)	Yes (real-time)	Yes (real-time)	Yes (hourly updates)
Spatial Coverage	High (distributed IoT nodes)	Low (fixed stations, sparse)	High (crowd-sourced)	Medium (purchased units)	Medium (regulatory + satellites)
Cost per Node	Low (~$50-100)	Very High ($10,000-50,000)	Low (~$200-300)	Medium ($250-400)	N/A (free service)
Pollutants Measured	PM2.5, PM10, CO_2, CO, VOCs, T, RH	PM2.5, PM10, NO_x, SO_2, O_3, CO	PM2.5, PM10	PM2.5, PM10, CO_2, VOCs	PM2.5, PM10, NO_2, O_3
Machine Learning Prediction	Yes (Random Forest, BLSTM)	No	No	Limited	Yes (Google AI)
Mobile Application	Yes (Flutter cross-platform)	Limited (web only)	Yes (iOS, Android)	Yes (iOS, Android)	Yes (integrated Maps/Search)
Pollution Hotspot Identification	Yes (K-Means clustering)	Manual analysis	Community-based	No	Limited
Personalized Health Alerts	Yes (user health profiles)	No	Basic threshold alerts	Yes	No
Route Planning	Yes (pollution-aware routing)	No	No	No	No
Community Reporting	Yes (crowd-sourced incidents)	No	Limited	No	No
Calibration/Accuracy	Moderate (periodic calibration)	High (reference-grade)	Variable (user-dependent)	Moderate	High (hybrid approach)
Scalability	High (cloud-native, modular)	Low (infrastructure-intensive)	High (crowd-sourced)	Medium	Very High (Google infrastructure)
Power Requirements	Low (5V DC, solar option)	High (continuous power)	Low (plug-in)	Low (plug-in, battery)	N/A

Feature	Proposed System	Government CPCB Stations	PurpleAir	Airveda	Google Air Quality
Maintenance Burden	Moderate (periodic calibration)	High (professional maintenance)	Low (user-managed)	Low (user-managed)	None (managed by Google)

Fig 7.1 Feature Comparison of our system with competitors

7.1 Comparative Analysis

Advantages Over Government CPCB Stations:

The proposed system provides vastly superior spatial coverage through distributed low-cost IoT nodes compared to expensive reference-grade stations. While government stations offer higher measurement accuracy, their sparse distribution (typically 1-2 stations per city) fails to capture neighborhood-level pollution variations critical for individual exposure assessment. Machine learning integration enables predictive capabilities absent in traditional monitoring networks.

Advantages Over Consumer Products (PurpleAir, Airveda):

Unlike commercial consumer solutions focusing solely on indoor or localized outdoor monitoring, the proposed system offers comprehensive pollution prediction (Random Forest, BLSTM), hotspot identification (K-Means clustering), and personalized health-based routing. The integrated mobile application provides actionable intelligence beyond simple measurement display. Open APIs and community reporting features foster ecosystem development impossible with closed commercial platforms.

Advantages Over Google Air Quality:

While Google leverages massive data infrastructure and sophisticated AI algorithms achieving excellent global coverage, the proposed system offers superior customization for local needs, transparent open-source algorithms (versus proprietary Google methods), and direct community engagement through reporting features. Local deployment maintains data sovereignty and enables regulatory agencies to directly control monitoring infrastructure.

Limitations Compared to Existing Systems:

Accuracy: Reference-grade government stations provide superior measurement precision critical for regulatory compliance and scientific research. The proposed system should be viewed as complementary rather than replacement.

Established Infrastructure: Platforms like Google Air Quality benefit from mature global infrastructure, extensive validation, and integration with widely-used services. The proposed system requires initial deployment effort and community building.

Professional Support: Commercial products offer customer support, warranty, and maintenance services. Open-source systems demand technical expertise for troubleshooting and

maintenance.

Strategic Positioning

The proposed system occupies a valuable niche providing **affordable, predictive, community-engaged air quality monitoring** balancing cost-effectiveness with advanced analytics. It serves populations and communities unable to afford expensive commercial solutions or underserved by sparse government networks, while offering transparency and customization impossible with proprietary platforms.

CHAPTER 8
TESTING

Testing is a crucial stage in the development of the IoT-based real-time pollution monitoring system, as it ensures that each component of the system operates correctly and that the integrated system performs reliably under real-world conditions. The testing process focused on validating the accuracy of sensor readings, verifying data communication between system components, evaluating backend functionality, and ensuring proper visualization of environmental data in the mobile application. Multiple testing strategies were implemented to examine both individual components and the overall system performance.

8.1 Hardware Testing

Hardware testing was performed to verify the proper functioning of the environmental sensors and their integration with the Arduino microcontroller. Each sensor was tested individually before integrating it into the complete system. The DHT22 sensor was tested to confirm its ability to measure temperature and humidity accurately. The MQ7 sensor was evaluated for detecting carbon monoxide levels, while the MQ135 sensor was tested for monitoring air quality indicators such as harmful gases and volatile compounds.

During the testing process, sensor outputs were observed using the serial monitor in the Arduino IDE. This allowed verification of whether the sensor readings were stable and within reasonable environmental ranges. Additionally, the sensors were allowed to warm up before testing to ensure accurate and stable measurements.

8.2 Communication Testing

Communication testing was conducted to ensure reliable data transmission between the Arduino microcontroller and the backend server. The Arduino device transmits environmental data in JSON format through serial communication using a USB connection.

The backend system continuously reads incoming serial data and parses the JSON messages containing sensor readings. During testing, multiple data transmission cycles were performed to verify that the backend could consistently receive and interpret the transmitted data without errors or interruptions. Error handling mechanisms were also tested to ensure that corrupted or incomplete data packets were ignored by the system.

This testing phase confirmed that the serial communication channel was stable and capable of transmitting environmental data in real time.

8.3 Backend System Testing

The backend system developed using Python and the Flask framework was tested to ensure proper data processing and API functionality. The backend was evaluated for its ability to receive sensor data, preprocess the readings, and generate pollution predictions using the machine learning model.

The REST API endpoints provided by the backend server were tested using HTTP requests to confirm that the system could return the latest environmental data in JSON format. The testing process verified that the backend correctly handled concurrent requests from the mobile application while maintaining accurate environmental readings.

Additionally, the background thread responsible for reading serial data was monitored to ensure that it could continuously process incoming sensor data without causing system delays or crashes.

8.4 Machine Learning Model Testing

The machine learning module was evaluated to verify its ability to predict pollution levels based on environmental parameters. The Random Forest classification model used in the system was trained using historical air quality datasets and evaluated using standard performance metrics.

The model's predictions were tested by providing sample environmental inputs such as temperature, humidity, carbon monoxide levels, and air quality sensor values. The output predictions were analyzed to ensure that the model produced meaningful classifications such as Good, Moderate, or Unhealthy air quality conditions. The evaluation demonstrated that the machine learning model was capable of providing reliable predictions, contributing to the system's ability to offer intelligent pollution analysis.

8.5 Mobile Application Testing

The Flutter-based mobile application was tested to verify that it could successfully communicate with the backend server and display environmental data correctly. The application periodically sends requests to the backend API to retrieve the latest sensor readings and pollution predictions.

During testing, the application interface was evaluated to ensure that temperature, humidity, carbon monoxide levels, and air quality values were displayed accurately. Real-time updates were monitored to confirm that the application refreshed environmental data without noticeable delays.

The alert system was also tested to verify that users receive notifications when pollution levels exceed predefined safety thresholds. This functionality ensures that users are promptly informed about potentially hazardous environmental conditions.

8.6 System Integration Testing

After verifying individual components, the complete system was tested to ensure proper integration

of hardware, backend processing, machine learning prediction, and mobile application visualization. The integrated system was operated continuously to observe real-time environmental monitoring and data transmission.

The results confirmed that the system successfully captured environmental data, transmitted it to the backend server, processed the information using machine learning algorithms, and displayed the results through the mobile application. The system maintained stable operation during continuous monitoring, demonstrating the reliability of the proposed pollution monitoring framework.

Through comprehensive testing of hardware, communication channels, backend processing, machine learning modules, and mobile application functionality, the system was validated as a reliable platform for real-time environmental pollution monitoring.

CHAPTER 9
RESULTS

The results of the proposed IoT-based real-time pollution monitoring system demonstrate the successful integration of hardware sensors, backend processing, machine learning prediction, and a mobile application interface. The system was evaluated based on hardware functionality, sensor data acquisition, machine learning prediction accuracy, and visualization of environmental parameters through the mobile application. The results confirm that the system is capable of collecting environmental data in real time, analyzing pollution conditions, and presenting meaningful insights to users.

9.1 Circuit Design and Hardware Setup

The hardware circuit of the proposed pollution monitoring system was designed by connecting multiple environmental sensors to an Arduino Uno microcontroller using a breadboard. The MQ7 carbon monoxide sensor and MQ135 air quality sensor were connected to the analog input pins of the Arduino board. These sensors detect harmful gases and provide analog voltage outputs corresponding to gas concentration levels in the environment.

The DHT22 sensor was connected to a digital input pin of the Arduino to measure environmental temperature and humidity. The Arduino board receives signals from these sensors and converts them into digital data using its built-in analog-to-digital converter.

The breadboard was used to simplify circuit connections and allow easy integration of the sensors with the Arduino microcontroller. Proper power supply connections were established to ensure stable sensor operation. Once the sensors were initialized, the Arduino continuously collected environmental readings and transmitted the data to the backend server through serial communication.

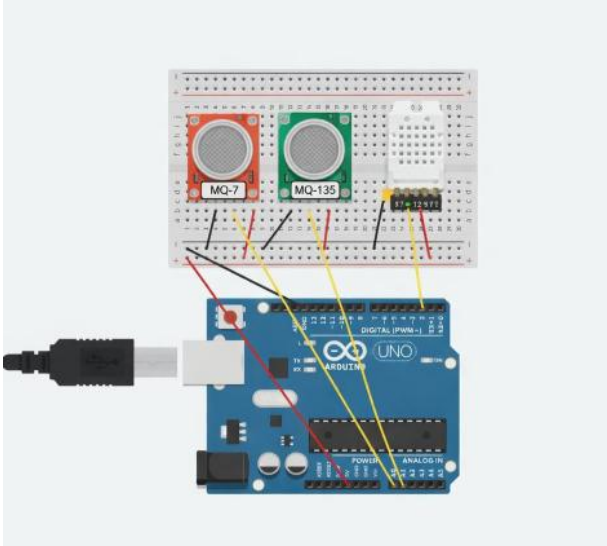

Fig 9.1 Circuit Design

9.2 Sensor Setup and Data Acquisition

The sensor setup represents the physical implementation of the pollution monitoring system. The MQ7 and MQ135 sensors were mounted on a breadboard along with the DHT22 sensor and connected to the Arduino Uno microcontroller using jumper wires.

The MQ7 sensor continuously monitors carbon monoxide levels in the environment, while the MQ135 sensor detects gases such as carbon dioxide and other air pollutants. The DHT22 sensor measures temperature and humidity, which are important environmental parameters that influence pollution levels.

During system operation, the Arduino reads data from the sensors at regular intervals and transmits the readings to the backend server. The collected data includes temperature, humidity, carbon monoxide concentration, and air quality values. These readings are formatted as JSON objects and sent to the backend server for processing and prediction.

The successful operation of the sensor setup confirms that the system can reliably collect environmental data for pollution monitoring.

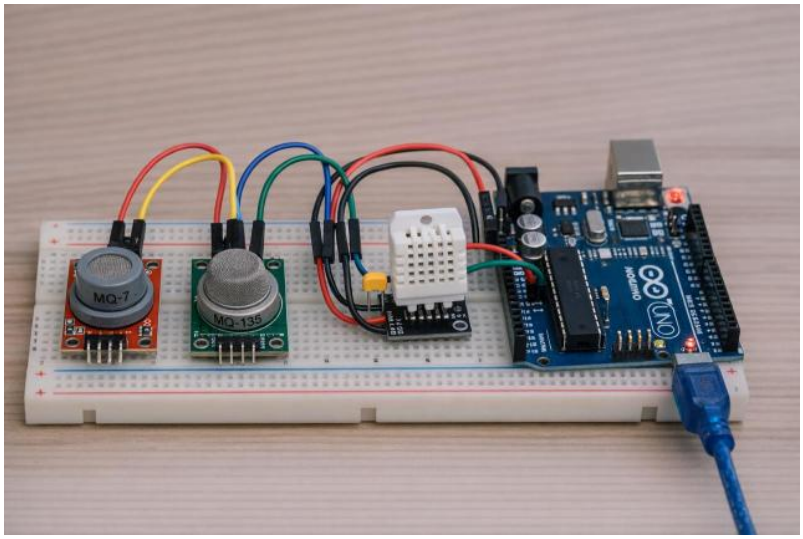

Fig 9.2 Sensor Setup

9.3 Mobile Application Interface

A cross-platform mobile application was developed using Flutter to provide users with an interactive interface for monitoring environmental conditions. The mobile application communicates with the backend server through API requests and retrieves real-time environmental data. The application includes several important user interface components. The login interface allows users to authenticate and access the monitoring system. After logging in, users can access the location-based monitoring feature that retrieves the user's geographic location and associates pollution data with that location. The mobile application displays environmental information such as temperature, humidity,

carbon monoxide levels, and air quality index values. These parameters are presented through graphical elements and numerical indicators that allow users to quickly understand environmental conditions.The application also includes a pollution analytics dashboard that highlights whether the current environment is safe or polluted. This feature provides users with immediate feedback regarding environmental safety.

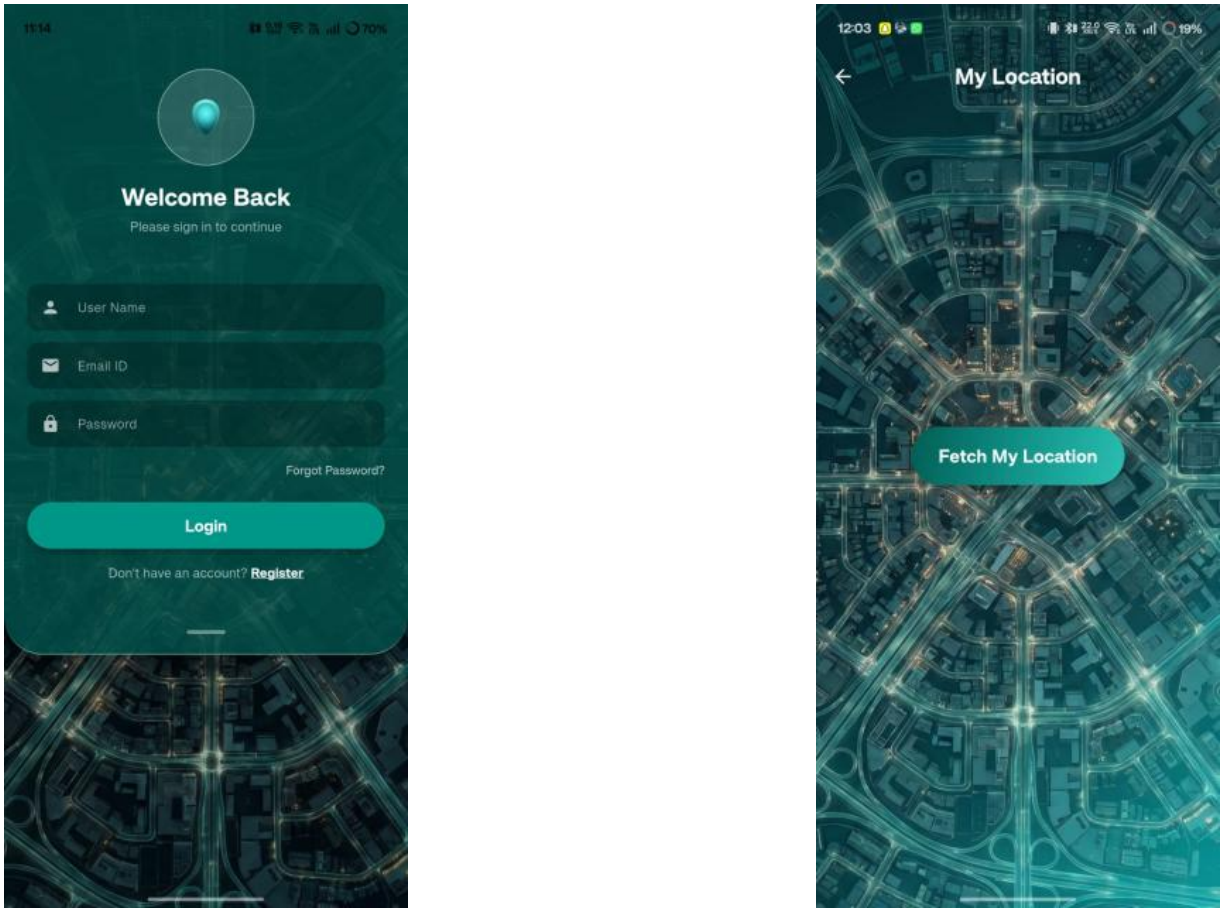

Fig 9.3 ***Mobile App Interfaces***

9.4 Real-Time Pollution Monitoring Dashboard

The pollution analytics dashboard presents real-time environmental data collected from the sensors. The dashboard displays key parameters including carbon monoxide concentration, air quality index values, temperature, and humidity.

In addition to displaying environmental readings, the system also provides a pollution status indicator. The application analyzes the sensor readings and determines whether the environment is safe or polluted. When pollution levels exceed predefined thresholds, the system displays warning messages to inform users about unsafe environmental conditions.

The dashboard continuously updates sensor readings as new data is received from the backend server. This real-time monitoring capability allows users to stay informed about changes in environmental conditions.

Fig 9.4 Real-Time Pollution Monitoring Dashboard

9.5 Machine Learning Model Performance

The performance of the machine learning model used in the system was evaluated using a classification approach. A Random Forest classification model was trained to predict pollution conditions based on environmental parameters such as temperature, humidity, carbon monoxide concentration, and air quality sensor values.

The trained model achieved an overall accuracy of 92.14%, indicating strong predictive performance. The evaluation was conducted using a dataset consisting of 7,455 environmental samples, which included both pollution-free and polluted conditions.

The confusion matrix of the model shows that the system correctly classified a large number of pollution-free and polluted instances. The model achieved a precision score of 0.91 and a recall score of 0.89 for the pollution-free class, while the polluted class achieved a precision of 0.93 and recall of 0.94. These results demonstrate that the model performs consistently across both classes.

Additionally, the macro and weighted F1-scores of the model were approximately 0.92, indicating balanced classification performance. The strong performance metrics confirm that the machine learning model is capable of accurately predicting pollution conditions based on sensor data.

```
Model Accuracy: 0.9214

Confusion Matrix:
 [[2558  318]
 [ 268 4311]]

Classification Report:
                    precision    recall  f1-score   support

Pollution-Free (0)       0.91      0.89      0.90      2876
      Polluted (1)       0.93      0.94      0.94      4579

          accuracy                           0.92      7455
         macro avg       0.92      0.92      0.92      7455
      weighted avg       0.92      0.92      0.92      7455
```

Fig 9.5 Model Performance

9.6 Overall System Performance

The results obtained from system evaluation demonstrate that the proposed pollution monitoring system operates successfully across all components. The hardware sensors were able to capture environmental data reliably, and the Arduino microcontroller successfully transmitted sensor readings to the backend server. The backend processing system efficiently analyzed the data and generated pollution predictions using the machine learning model. The Flutter mobile application successfully retrieved the processed data and displayed real-time environmental conditions to users. Overall, the system provides an effective and low-cost solution for monitoring air pollution and promoting environmental awareness through real-time data visualization and intelligent pollution prediction.

CHAPTER 10
CONCLUSION

This project presents the development of an IoT-based real-time pollution monitoring system designed to provide continuous environmental monitoring and predictive analysis of air quality conditions. The system integrates environmental sensors, an Arduino microcontroller, a Python-based backend server, and a Flutter mobile application to create a complete end-to-end monitoring platform.

The system successfully collects environmental parameters such as temperature, humidity, carbon monoxide levels, and air quality indicators using low-cost sensors. These readings are transmitted to a local server where they are processed and analyzed using machine learning algorithms. The Random Forest classification model provides predictive insights regarding pollution levels and environmental safety.

The integration of a mobile application allows users to easily access real-time environmental data and pollution predictions. This enables individuals to make informed decisions regarding their daily activities and avoid exposure to harmful air conditions.

Despite the effectiveness of the system, certain limitations exist. Low-cost sensors may require calibration to improve measurement accuracy, and the current system relies on a local server rather than a cloud-based infrastructure. Additionally, the system currently monitors a limited number of pollutants.

Future improvements may include integrating additional sensors for particulate matter detection, deploying the system on cloud platforms for large-scale monitoring, and enhancing prediction models using advanced deep learning techniques. These enhancements would further improve the accuracy, scalability, and usability of the pollution monitoring system.

CHAPTER 11
FUTURE SCOPE

The proposed IoT-based pollution monitoring system can be further enhanced in several ways to improve its functionality and scalability. One possible improvement is the integration of additional sensors such as PM2.5 and PM10 sensors to measure particulate matter and provide a more comprehensive analysis of air quality. This would allow the system to monitor a wider range of pollutants.

Another potential enhancement is the deployment of the backend system on cloud platforms. Cloud integration would enable large-scale data storage, improved system reliability, and access to environmental data from multiple locations. It would also allow long-term data analysis and large-scale environmental monitoring.

The machine learning module can also be improved by incorporating advanced models such as deep learning algorithms to provide more accurate pollution predictions. These models could analyze historical pollution data to predict future air quality trends.

Additionally, the mobile application can be enhanced by integrating interactive maps and personalized alerts that notify users when pollution levels become unsafe. These improvements would make the system more useful for public awareness and environmental monitoring.

With these enhancements, the system can evolve into a more intelligent and scalable platform for real-time air quality monitoring and smart city applications.

REFERENCES

[1] Bobulski, J., Szymoniak, S., & Pasternak, K. (2024). An IoT System for Air Pollution Monitoring with Safe Data Transmission. Sensors, 24(2), 445.

[2] Y.-L. Chiang, J.-C. Wang, M.-H. Lee, A.-C. Liu, and J.-A. Jiang, "Deep-Learning-Based Multi-Timestamp Multi-Location PM2.5 Prediction: Verification by Using a Mobile Monitoring System With an IoT Framework Deployed in the Urban Zone of a Metropolitan Area," *IEEE Internet of Things Journal*, vol. 11, no. 5, pp. 8815–8836, Mar. 2024.

[3] F. Naz, M. Fahim, A. A. Cheema, B. D. E. McNiven, T.-V. Cao, R. Hunter, and T. Q. Duong, "Air Quality and Healthy Ageing: Predictive Modeling of Pollutants Using CNN Quantum-LSTM," *IEEE Access*, vol. 13, pp. 94212–94227, May 2025.

[4] M. N. A. Ramadan, M. A. H. Ali, S. Y. Khoo, M. Alkhedher, and M. Alherbawi, "Real-time IoT-powered AI System for Monitoring and Forecasting of Air Pollution in Industrial Environment," *Ecotoxicology and Environmental Safety*, vol. 283, p. 116856, Aug. 2024.

[5] H.-C. Hsu, G. B. Tarekegn, M. E. Getnet, K.-P. Yu, and L.-C. Tai, "UAV-Based Real-Time Air Quality Monitoring and Prediction Using Machine Learning Approach," *IEEE Internet of Things Journal*, vol. 12, no. 14, pp. 27915–27928, Jul. 2025.

[6] https://www.kaggle.com/datasets/abhisheksjha/time-series-air-quality-data-of-india-2010-2023